

Programming in Java Project
Semester: IV Batch: 2024-28
AY 2025-26

PIJ Final Project Presentation

GROUP NO- 53

TITLE OF PROJECT- RETAILRISK AI

Strategic Client Identification and Risk-Based Prioritization
for Direct Marketing Campaigns in Retail Banking

NAME OF MENTOR- Prof. Mayur Gaikwad

GROUP MEMBERS:

Anushka Gupta – 24070126037 (A2)

Bhavya Tanna – 24070126046 (A2)

Dakshesh Chauhan – 24070126048 (A3)

Adityadev Sinha – 24070126228 (C3)

INTRODUCTION

Background & Real-World Context

Retail banks market

- Loans, Credit Cards
- Insurance, Investments

Mass marketing leads to

- High operational cost
- Increased financial risk
- Low conversion efficiency

Core business need

- Prioritise the right customers

Domain

- Retail Banking
- Direct Marketing
- Risk-Based Decision Support

Enterprise Reality

- Python builds ML models
- Java runs banking systems
- Java consumes predictions
- No ML training in Java

“This project addresses the missing decision layer that transforms raw probabilities into actionable, risk-aware marketing decisions.”

PROBLEM STATEMENT

From Data to Decisions: Smarter Banking Campaigns

The Challenge – Blind Outreach

- “Spray and pray” marketing campaigns
- Calls sent without customer prioritisation
- High cost, low conversion, customer fatigue

The Bottleneck – Stranded Intelligence

- Predictive ML models already exist
- Insights remain unused in operations
- High-risk customers treated equally

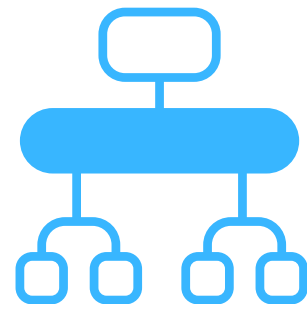
The Gap – Missing Decision Layer

- No system to convert probabilities into actions
- Lack of risk-aware customer ranking

Objective

- Develop a Java-based decision system
- Consume ML probability scores from Python
- Enable targeted, risk-aware campaign execution

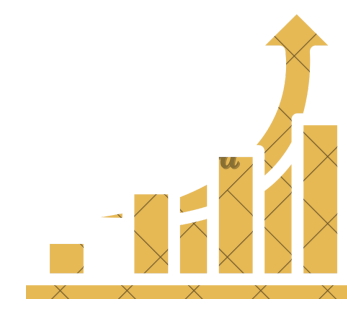
OBJECTIVES



Predict customer
likelihood of term
deposit subscription



Convert probability
into actionable
decisions



Develop Java-based
decision engine

PROPOSED SOLUTION

Two-Layer Smart Architecture

Layer 1 – The Brain (Python ML Module)

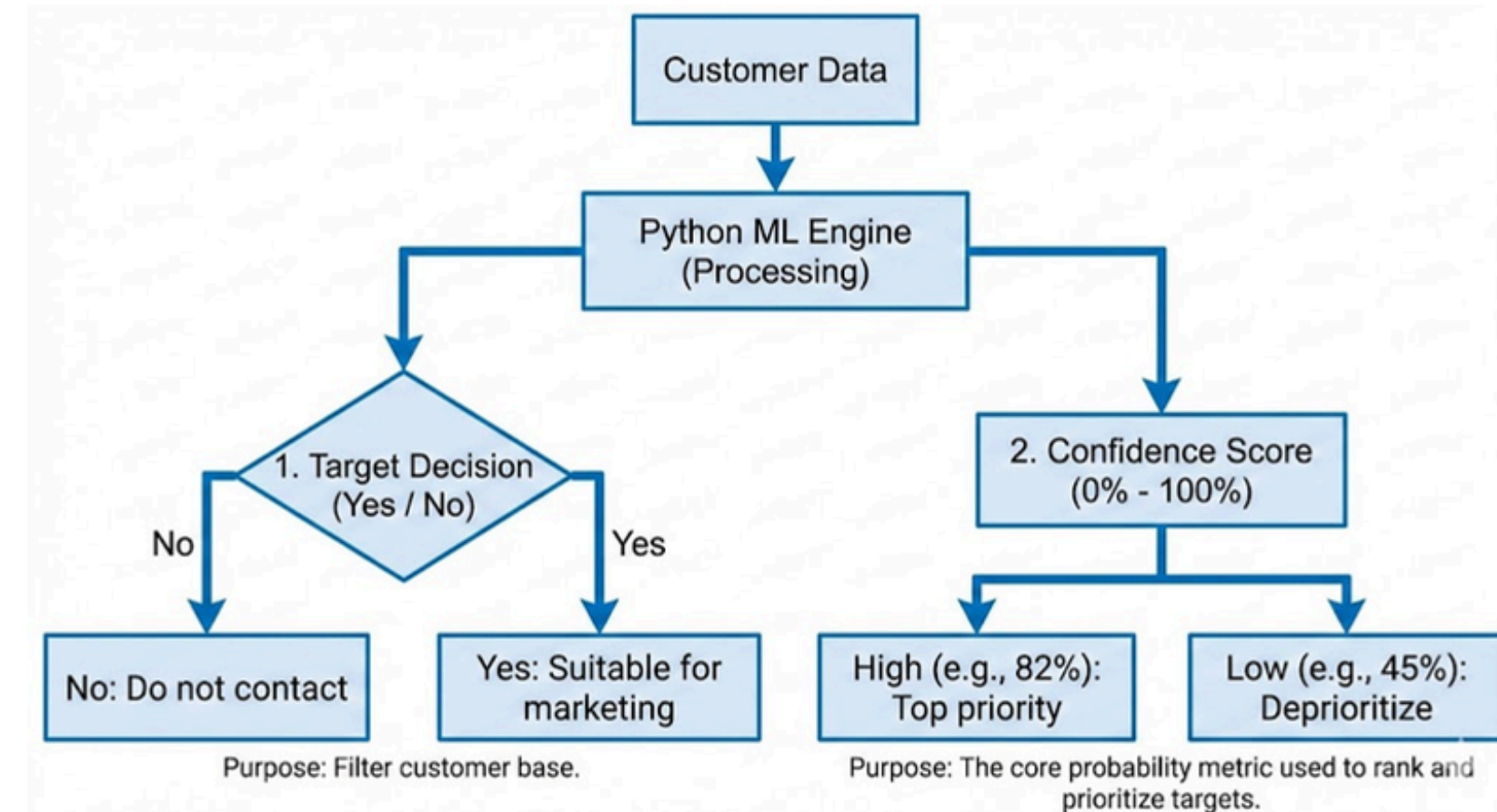
- Analyzes customer financial & behavioral data
- Generates Decision Output (Yes / No)
- Produces Confidence Score (0–100%)
- Enables customer ranking based on probability

Layer 2 – The Muscle (Java Application Module)

- Consumes ML predictions via REST API
- Applies
 - Business rules & risk evaluation
 - Tier-based classification (High / Moderate / Low)
- Uses OOP + Factory Pattern to:
 - Generate personalized marketing strategies
 - Handle dynamic decision logic

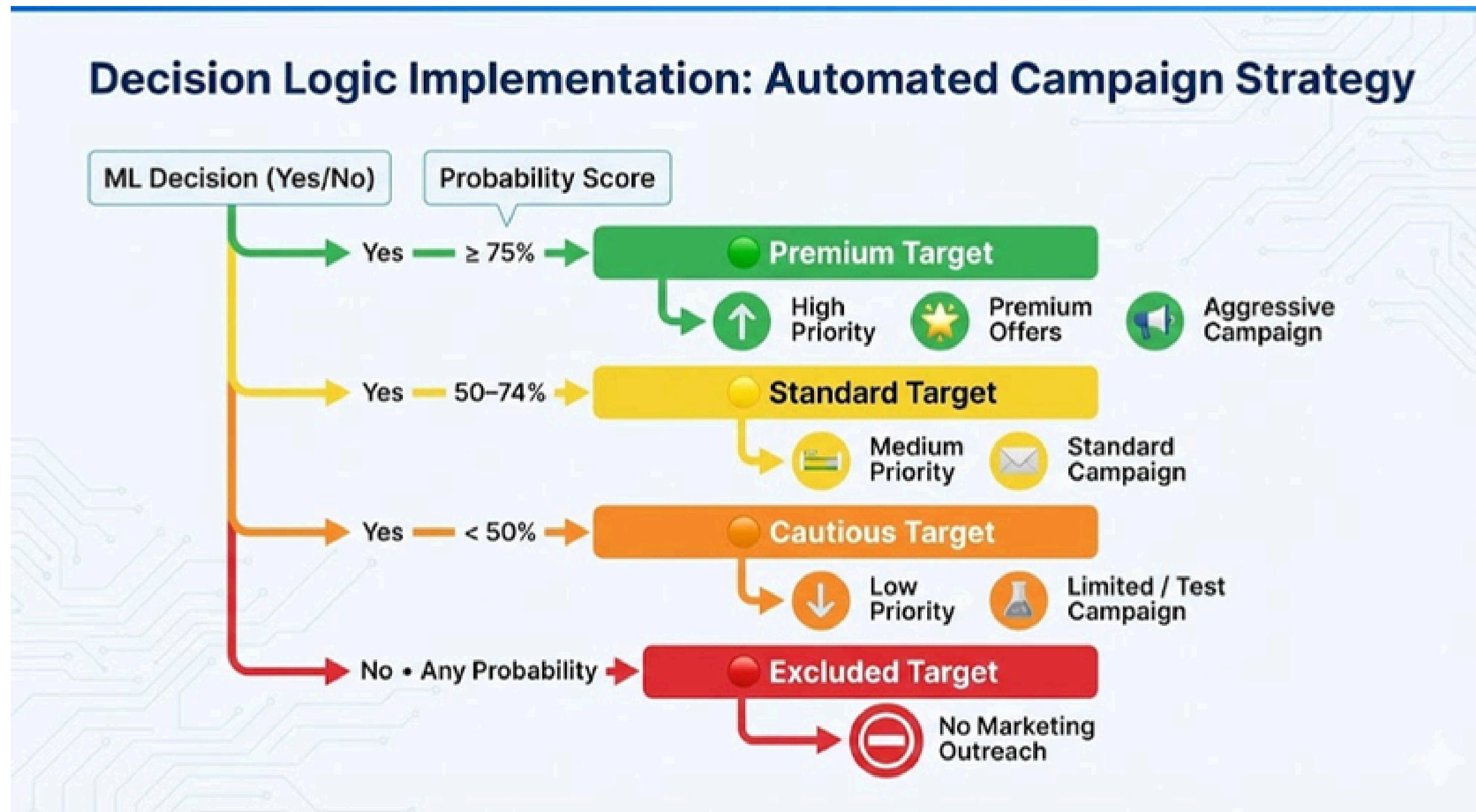
System Outcome

- User uploads data → Java Backend → ML API → Prediction → Java Decision Engine → Database Storage → UI Display
- Cloud deployment (Render + TiDB Cloud)



HOW JAVA USES BOTH PYTHON OUTPUTS

“Java acts as the decision execution engine. While Python predicts probability, Java translates those probabilities into structured campaign strategies using Object-Oriented rule design. This separation ensures scalability, maintainability, and enterprise readiness.”



OOP DESIGN OVERVIEW

Real-World to OOP Mapping

- ML Output → PredictionResult (DTO)
- Business Logic → MarketingPolicy (Abstract Class)
- Strategy Creation → PolicyFactory (Factory Pattern)
- Service Layer → RetailRiskService
- Database Layer → CustomerPrediction (Entity)
- UI Interaction → Controller + Thymeleaf UI

Why OOP Is Mandatory

- Handles multiple customer tiers dynamically
- Enables flexible marketing strategies
- Reduces code duplication
- Maintains clean separation of concerns
- Supports scalability and future enhancements

ENCAPSULATION

What Encapsulation Means Here

- Customer and prediction data are secured using private fields
- Includes:
 - Probability score
 - Tier classification
 - Customer attributes
- Access is controlled through methods only

How It Is Implemented

- Classes
 - CustomerPrediction.java (Entity)
 - PredictionResult.java (DTO)

All fields are declared private with controlled access through public getters and setters. Internal database mapping (@Entity, @Table, @Column) is hidden from external code.

Why This Is Important

- Prevents direct data modification
- Protects ML prediction integrity
- Ensures secure database interaction
- Maintains clean separation of data and logic

“Encapsulation ensures controlled and secure access to customer and prediction data.”

INHERITANCE

Inheritance – Why It Is Used

- Different priority customers require
 - Different campaign behaviour
 - Different offer limits
 - Different messaging strategies

Problem Without Inheritance

- Repeated code for each customer tier
- Complex conditional (if-else) logic
- Difficult to maintain and extend

Inheritance Structure

- Abstract Class: MarketingPolicy
 - Subclasses: HighTierPolicy
 - ModerateTierPolicy
 - LowTierPolicy
- Each subclass defines its own:
 - generateRecommendation()
 - Tier-specific behavior

How Python Output Drives Inheritance

- Flask ML API returns:
 - Probability score
 - Likelihood tier
- Java uses PolicyFactory to:
 - Read the tier
 - Create appropriate subclass
- Example:
 - High tier → HighTierPolicy
 - Moderate → ModerateTierPolicy

Benefits

- Code reusability through base class
- Clean and scalable class hierarchy
- Easy to add new tiers or strategies
- Separation of business logic

“Inheritance allows probability-driven specialization of customer behaviour without duplicating code.”

POLYMORPHISM

Polymorphism – How It Works

What Polymorphism Does Here

- Same method: generateRecommendation()
- Different behavior based on policy type
- Method defined in abstract class MarketingPolicy
- Implemented differently in each subclass

Example Runtime Behaviour

- *HighTierPolicy* → *Premium / aggressive strategy*
- *ModerateTierPolicy* → *Nurture campaign strategy*
- *LowTierPolicy* → *Low-cost awareness strategy*

Method call

`policy.generateRecommendation();`

How is it achieved

- PolicyFactory creates object based on ML output
- Returns type: MarketingPolicy
- Actual object:
 - HighTierPolicy / ModerateTierPolicy / LowTierPolicy

Dynamic dispatch happens at runtime

Why This Is Powerful

- Eliminates large if-else conditions
- Enables runtime decision-making
- Makes system scalable and flexible
- Easy to add new policy types

“Polymorphism allows probability-based dynamic decision making.”

ABSTRACTION

The Problem

- Business logic may evolve over time:
 - Changing probability thresholds
 - New marketing strategies
 - Updated campaign policies
- Direct implementation increases complexity

Abstraction Solution

- Abstract class: MarketingPolicy
- Defines common structure for all policies
- Key Abstract Methods:
 - generateRecommendation()
 - getTierDisplayName()
 - getTierCssClass()

Implementation details hidden from main application

How It Works

- MarketingPolicy defines what to do
- Subclasses define how to do it:
 - HighTierPolicy
 - ModerateTierPolicy
 - LowTierPolicy

Java interacts with abstract type, not concrete classes

Benefits

- Decouples business logic from application flow
- Easy to modify or extend rules
- Reduces system complexity
- Improves maintainability and scalability

“Abstraction decouples risk evaluation logic from application flow, enabling independent rule evolution.”

User Interaction (UI) Module

Type of UI

- Web-based UI using Thymeleaf (Spring Boot)
- Responsive interface (HTML, CSS, JavaScript)
- Drag-and-drop CSV upload support
- Scalable to REST API

UI Workflow

1. User uploads CSV or enters customer data
2. Spring Boot sends data to ML microservice (Flask API)
3. ML service returns prediction
4. Backend processes results using Decision Engine
5. UI displays:
 - Prediction (Yes / No)
 - Probability score
 - Tier classification
 - Marketing recommendation

UI Responsibilities

- Drag-and-drop file upload interface
- Ranked results table with probability scores
- Tier-based color coding (High / Moderate / Low)
- Probability bar visualization
- Responsive design with modern layout

OOP Principles in UI

- Encapsulation: UI does not access internal data directly
- Abstraction: UI interacts only with Service/Controller layer
- Polymorphism: Same UI handles multiple policy outputs
- Loose Coupling: UI independent of ML and business logic

“The UI acts purely as an interaction layer, ensuring clean separation from decision logic and machine learning.”

CRUD Functionality in System

- The system supports full Create, Read, Update, Delete (CRUD) operations on customer prediction data

Create

- Upload CSV or enter customer data
- ML model generates:
 - Probability score
 - Tier classification
 - Recommendation

Read

- View all customer predictions in a ranked table
- Displays:
 - Probability
 - Tier
 - Campaign recommendation

Update

- Modify customer details
- Automatically triggers re-prediction using ML API
- Updates tier and recommendation dynamically

Delete

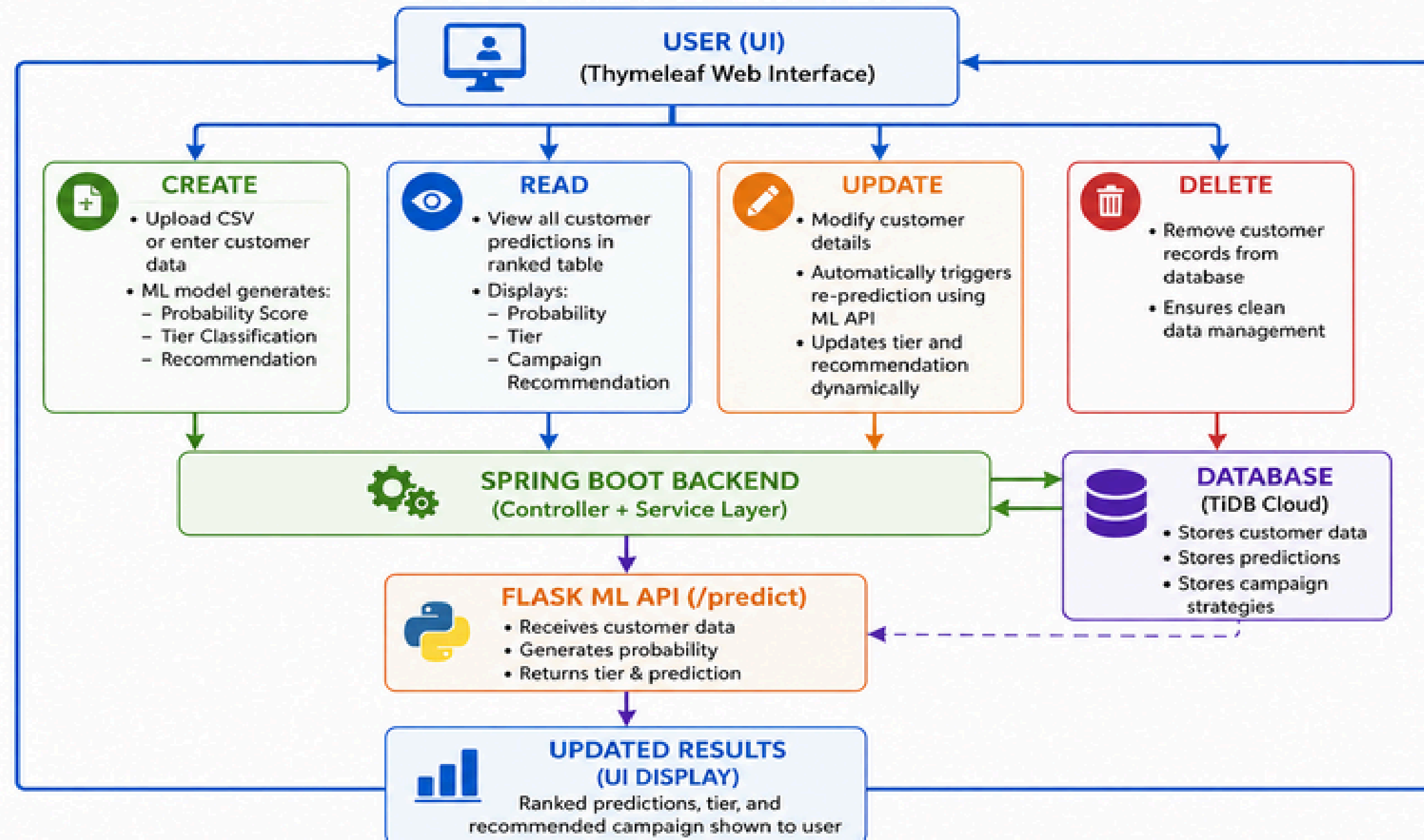
- Remove customer records from database
- Ensures clean data management

Key Feature

System integrates CRUD operations with ML, enabling dynamic re-evaluation of predictions.

CRUD OPERATIONS FLOW

End-to-End Flow of Data in RetailRisk AI System



KEY HIGHLIGHT

Create and Update operations trigger real-time ML predictions. Read retrieves stored data. Delete removes records.

System Development Phases

1. Problem Analysis & Requirement Definition

- Defined need for ML-driven decision support system
- Finalized outputs:
- Probability score, Tier classification, Marketing recommendation

2. ML Model Development (Python)

- Data preprocessing and feature engineering
- Model training using XGBoost, Logistic regression
- Output generation:
 - Probability score
 - Tier classification
 - Marketing recommendation

3. Backend Development (Java – Spring Boot)

- Developed enterprise web application
- Integrated ML API using REST calls
- Implemented:
 - Decision Engine
 - PolicyFactory (Factory Pattern)
 - CRUD operations using JPA/Hibernate
- Business logic for campaign recommendations

4. Database Integration (TiDB Cloud)

- Cloud-based MySQL-compatible database
- Stores:
 - Customer data
 - Predictions
 - Rankings & recommendations
- Ensures persistence and re-prediction

5. UI Development (Web Interface)

- CSV upload (drag-and-drop)
- Display ranked predictions
- CRUD operations (view/edit/delete)
- Real-time re-prediction on update

6. Deployment & Integration

- ML service deployed on Render (Flask)
- Java application deployed via Docker (Spring Boot)
- Communication via REST API (JSON)
- Secure DB connection (TLS-enabled)

Tools & Technologies Used

Programming Languages

- Python – Machine Learning model development
- Java – Enterprise application and decision logic

Libraries / Frameworks

- Python: pandas, numpy, scikit-learn
- Java: Core Java (OOP, Collections)
- Flask – ML microservice (REST API)
- Spring Boot – Backend web application

IDE & Development Tools

- Python IDE: Jupyter Notebook / PyCharm / VSCode
- Java IDE: IntelliJ IDEA / VSCode

FRONTEND

- HTML, CSS, JS

Version Control & Collaboration

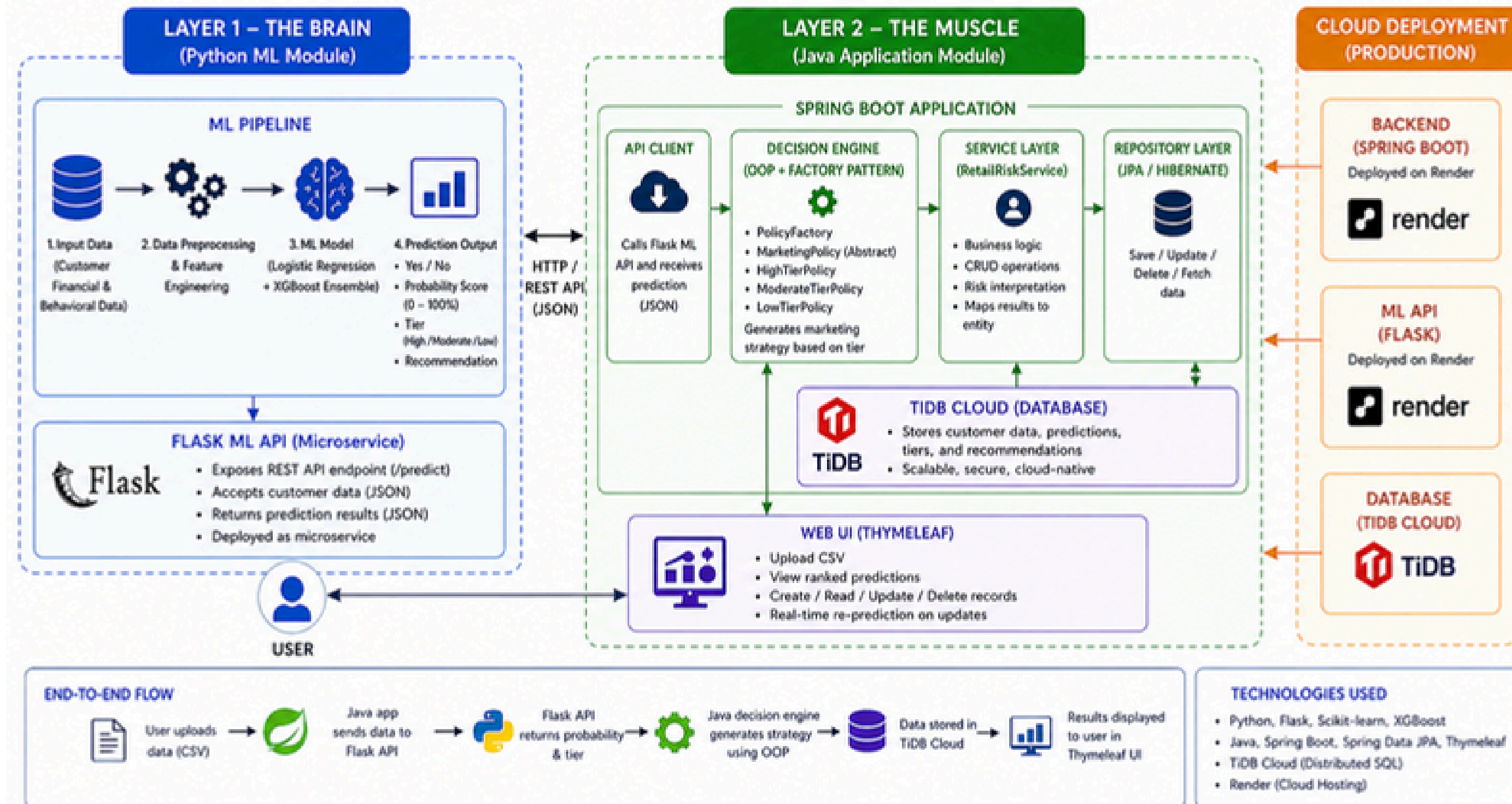
- Git used for source code management
- GitHub used for version control and backup
- Separate modules maintained clearly
- Commit-based development tracking

GitHub Repository

<https://github.com/Add14/PBL-1>

SYSTEM ARCHITECTURE

RetailRisk.ai – Two-Layer Smart Architecture



MODULE BREAKDOWN

Responsibility Allocation

Module 1: Machine Learning Engine (Python)

- Data preprocessing & feature engineering
- Model training
- Output generation (Probability, Tier, Recommendation)
- Assigned to: ADITYADEV SINHA

Module 2: Data Exchange Module

- REST API communication (Flask)
- JSON data transfer
- Data validation & consistency checks
- Python–Java integration layer
- Assigned to: DAKSHESH CHAUHAN

Module 3: Decision Engine & Business Logic (Java)

- Business logic & risk evaluation
- Tier-based classification
- PolicyFactory for strategy generation
- CRUD operations using JPA/Hibernate
- Assigned to: BHAVYA TANNA

Module 4: User Interface & Integration

- Web-based UI
- CSV upload (drag-and-drop)
- Display ranked predictions
- Edit / delete / update records
- Assigned to: ANUSHKA GUPTA

USER INTERFACE (UI)

 retailrisk.ai

BETA - INTERNAL USE ONLY

TERM DEPOSIT PROPENSITY ENGINE

Drop a CSV. Get *Ranked Results*.

Upload your customer dataset and the LR + XGBoost ensemble will score every row, tier them by likelihood, and generate a targeted marketing policy for each.

 Upload Customer CSV

Drag and drop or click to browse — multiple rows supported

REQUIRED CSV COLUMNS

age, job, marital, education, default, balance, housing, loan, contact, day, month, duration, campaign, pdays, previous, poutcome

Example row:

42, management, married, tertiary, no, 1500, yes, no, cellular, 15, may, 300, 1, -1, 0, unknown



Drop your CSV file here

or click to browse · .csv files only · up to 32 MB

USER INTERFACE (UI)

Customer ranked list


BETA - INTERNAL USE ONLY

Customer Results – Ranked by Probability

bank.csv

#	CUSTOMER	AGE / JOB	LR %	XGB %	FINAL PROB	TIER	VERDICT	POLICY RECOMMENDATION
1	Customer 2686	45 / admin	100.0%	72.7%	78.2%	• Likely	✓ Subscribe	View Policy
2	Customer 3790	47 / management	100.0%	72.4%	77.9%	• Likely	✓ Subscribe	View Policy
3	Customer 37	78 / retired	100.0%	72.2%	77.8%	• Likely	✓ Subscribe	View Policy
4	Customer 1159	38 / management	100.0%	72.2%	77.8%	• Likely	✓ Subscribe	View Policy
5	Customer 1089	39 / management	100.0%	72.1%	77.7%	• Likely	✓ Subscribe	View Policy
6	Customer 3386	38 / unknown	100.0%	72.1%	77.7%	• Likely	✓ Subscribe	View Policy
7	Customer 2422	44 / management	100.0%	71.8%	77.5%	• Likely	✓ Subscribe	View Policy
8	Customer 2774	35 / management	100.0%	71.8%	77.4%	• Likely	✓ Subscribe	View Policy
9	Customer 3312	87 / retired	100.0%	71.7%	77.4%	• Likely	✓ Subscribe	View Policy
10	Customer 856	46 / management	100.0%	71.7%	77.3%	• Likely	✓ Subscribe	View Policy
11	Customer 1111	75 / retired	100.0%	71.7%	77.3%	• Likely	✓ Subscribe	View Policy

USER INTERFACE (UI)

Campaign Suggestion


retailrisk.ai

BETA - INTERNAL USE ONLY

#	CUSTOMER	AGE / JOB	LR %	XGB %	FINAL PROB	TIER	VERDICT	POLICY RECOMMENDATION
								View Policy + PRIORITY ACTION — High Conversion Probability: 78.20% 1. IMMEDIATE OUTREACH: Assign a dedicated Relationship Manager to contact this customer within 24 hours via their preferred channel. 2. PERSONALISED OFFER: Present a tailored term deposit package with a competitive interest rate (offer 0.25–0.50% above standard rates) and flexible lock-in periods (3, 6, or 12 months). 3. PREMIUM INCENTIVES: Offer a welcome bonus for deposits above ₹1,00,000 — e.g., complimentary locker facility or loyalty reward points. 4. DIGITAL FAST-TRACK: Send a pre-filled digital application link via email and SMS. Reduce friction with one-click confirmation. 5. FOLLOW-UP: Schedule a callback within 48 hours if no response. Escalate to senior banker if the first contact does not convert.
1	Customer 2696	45 / admin.	100.0%	72.7%	78.2%		Likely Subscribe	
3	Customer 3790	47 / management	100.0%	72.4%	77.9%		Likely Subscribe	View Policy +

TIMELINE

Phase	Phase Name	Key Activities	Status
Phase 1	Planning & Design	Problem analysis, architecture design, OOP structure finalisation	Completed
Phase 2	ML Model Development (Python)	Data preprocessing, model training, probability output generation	Completed
Phase 3	Decision Engine Development (Java)	Risk evaluation logic, campaign rules, OOP implementation	Completed
Phase 4	UI & System Integration	Web-based UI, data exchange integration, end-to-end testing	Completed

CONCLUSION

RetailRisk AI demonstrates

- Full-stack ML + Java enterprise integration using microservices
- Deployment of ML model as a Flask-based REST API
- Spring Boot application for decision-making and user interaction
- Probability-driven tier-based customer prioritization system
- Implementation of OOP principles with Factory Pattern
- Scalable, cloud-deployed architecture (Render + TiDB Cloud)

Key Takeaway

- Machine learning becomes valuable only when integrated into real systems
- This project transforms predictions into actionable business decisions
- Demonstrates industry-level architecture and practical implementation

“This project bridges the gap between machine learning predictions and enterprise decision-making using Object-Oriented Java architecture.”

THANK YOU